



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Ejercicios VHDL

Silverio Martínez Andrés

Lab. Organización y Arquitectura de
Computadoras

Grupo: 8

Semestre 2026-2

Profesor: Jorge Ferat Toscano

Fecha de entrega: 11 de abril de 2026

Ejercicios VHDL

CÓDIGO:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity P4 is
  Port (A : in STD_LOGIC_VECTOR (2 downto 0);
        LUZ_ROJA: out STD_LOGIC;
        LUZ_VERDE: out STD_LOGIC);
end P4;

architecture Behavioral of P4 is
begin
  process(A)
  begin
    case A is
      when "000" =>
        Luz_Roja <= '0'; Luz_Verde <= '1';
      when "001"=>
        Luz_Roja <= '0'; Luz_Verde <= '1';
      when "010" =>
        Luz_Roja <= '0'; Luz_Verde <= '1';
      when "011"=>
        Luz_Roja <= '0'; Luz_Verde <='1';
      when "100"=>
        Luz_Roja <= '0'; Luz_Verde <= '1';
      when "101"=>
        Luz_Roja <= '1'; Luz_Verde <= '0';
      when "110"=>
        Luz_Roja <= '1'; Luz_Verde <= '0';
      when others =>
        Luz_Roja <= '1'; Luz_Verde <= '0';
    end case;
  end process;

end Behavioral;
```

Este código VHDL describe **un circuito lógico secuencial simple** que actúa como un **controlador de dos luces (roja y verde)**, cuyo estado depende del valor de una **entrada de 3 bits (A)**.

Este circuito **controla dos luces (roja y verde)** en función de una señal de **entrada de 3 bits (A)**.

- Si el valor de A está **entre 0 y 4**, la **LUZ_VERDE** se enciende.
- Si el valor de A está **entre 5 y 7**, la **LUZ_ROJA** se enciende.

Es decir, el diseño **actúa como un decodificador de rango o control lógico de estado**, muy útil en prácticas de control digital o simulaciones de semáforos.

COMPILACIÓN:

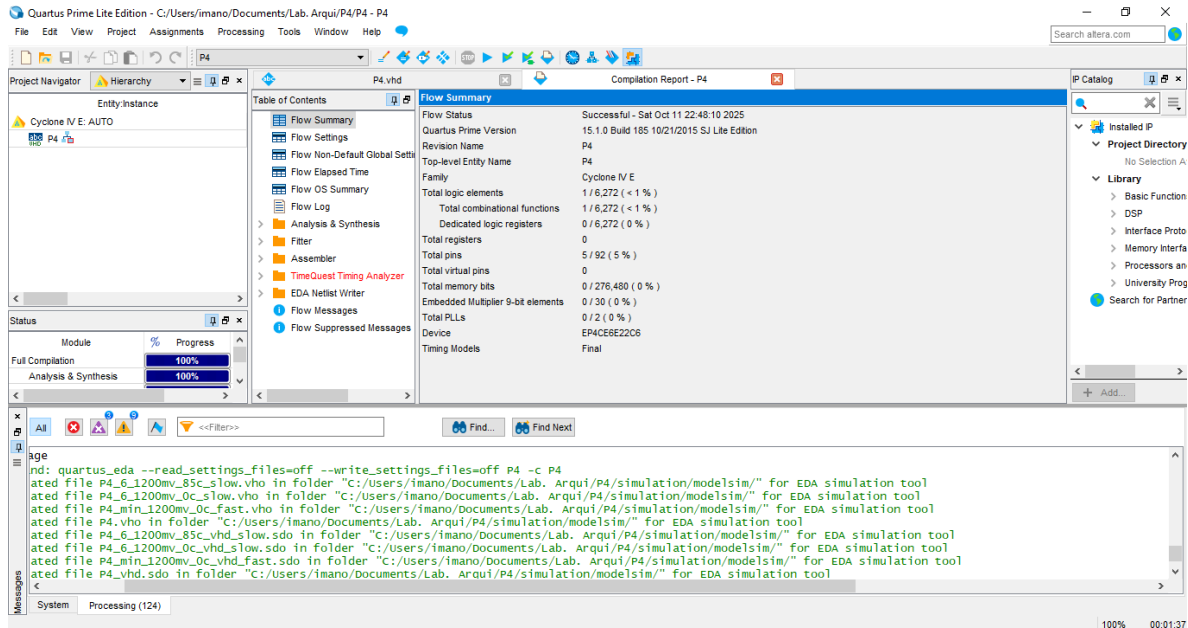


Imagen: Compilación

SIMULACIÓN 1

Como quiero hacer la simulación en Waveform Editor, entonces seguimos el siguiente paso. En la ventana New, en la sección Verification/Debugging Files seleccionar la opción University Program VWF.

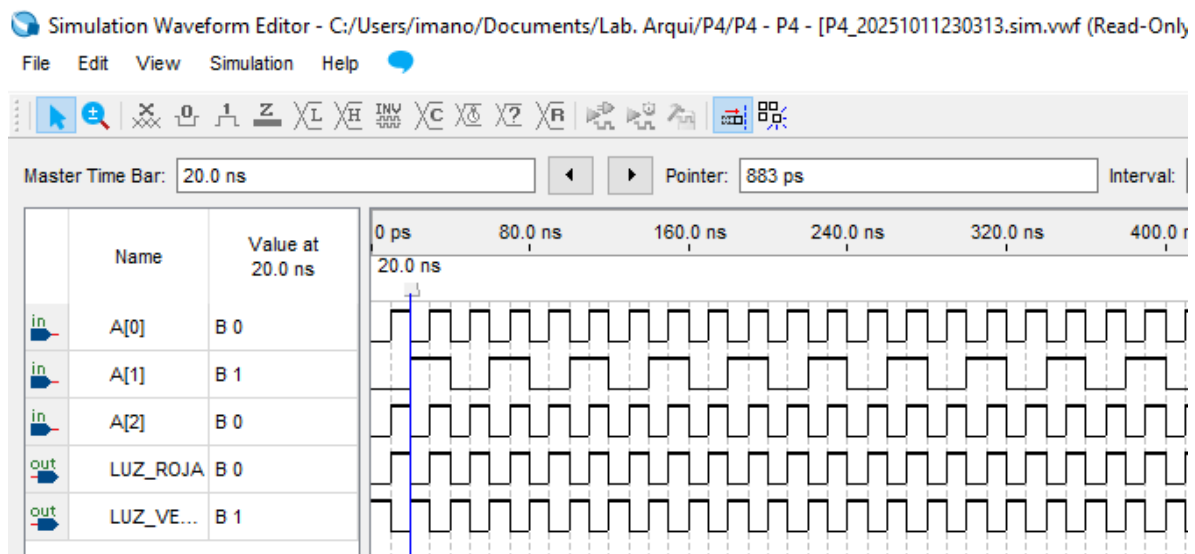


Imagen: Simulación

- A(0) → Bit menos significativo (A0)
- A(1) → Bit medio (A1)
- A(2) → Bit más significativo (A2)

Como ves, el patrón **se repite cada 40 ns**. La salida **LUZ_VERDE** **parpadea alternando con LUZ_ROJA** cada 10 ns aproximadamente. Esto se debe a que A0 y A2 cambian al mismo ritmo, generando saltos directos entre combinaciones “verdes” (000,010,100) y “rojas” (101,111).

Resultado general:

- La salida **LUZ_VERDE** se activa cuando A corresponde a valores entre 000 y 100.
- La salida **LUZ_ROJA** se activa cuando A es 101, 110 o 111.
- Debido a los periodos dados, el circuito alternará entre ambas luces de forma **rápida y no lineal**, con un **patrón repetitivo de 40 ns**.

SIMULACIÓN 2

Utilizamos la otra simulación que es la misma interpretación

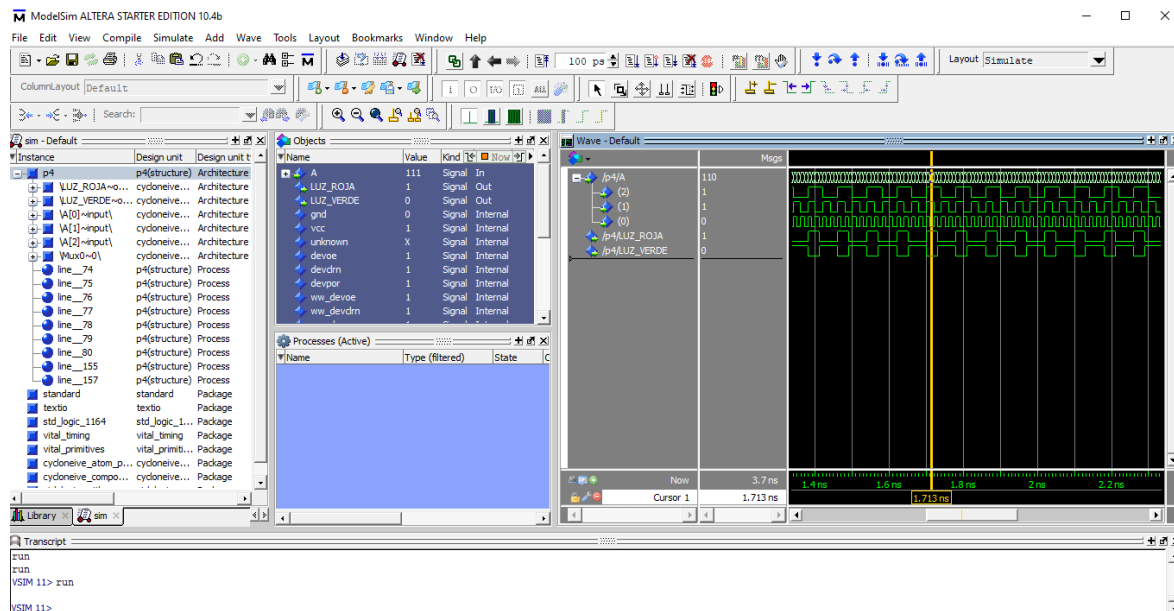


Imagen: Simulación

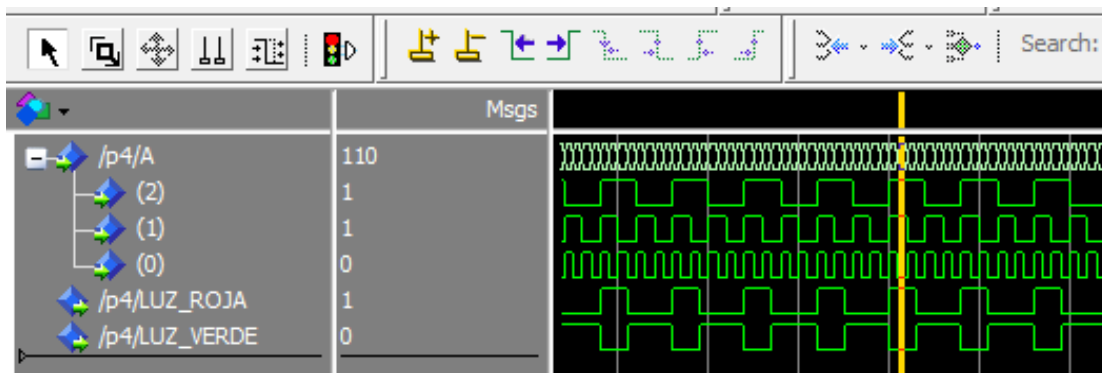


Imagen: Simulación

El código hace lo siguiente:

A2	A1	A0	A (binario)	LUZ_VERDE	LUZ_ROJA	Explicación
0	0	0	000	1	0	Verde encendida, caso inicial
0	0	1	001	1	0	Sigue verde
0	1	0	010	1	0	Sigue verde
0	1	1	011	1	0	Sigue verde
1	0	0	100	1	0	Último caso con verde
1	0	1	101	0	1	Cambio a rojo
1	1	0	110	0	1	Rojo encendida
1	1	1	111	0	1	Rojo encendida

- Cuando **A < 101** → LUZ_VERDE = 1, LUZ_ROJA = 0
- Cuando **A ≥ 101** → LUZ_VERDE = 0, LUZ_ROJA = 1

En otras palabras, la **LUZ_VERDE** se enciende en los primeros cinco estados binarios (0 a 4), y la **LUZ_ROJA** se enciende en los tres últimos (5 a 7).

CÓDIGO:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity P4 is
  Port ( A : in STD_LOGIC_VECTOR (1 downto 0);
        sal : out STD_LOGIC_VECTOR (6 downto 0));
end P4;

architecture Behavioral of P4 is begin
process (A)
begin
  case A is
    when "00" => sal <= "0001001";
    when "01" => sal <= "1000000";
    when "10" => sal <= "1000111";
    when "11" => sal <= "0001000";
    when others => sal <= "0000000";
  end case;
end process;
end Behavioral;
```

Este código está escrito en VHDL y define un pequeño decodificador que convierte una entrada binaria de 2 bits en una salida de 7 bits, probablemente destinada a controlar un display de 7 segmentos (como los usados en prácticas de circuitos digitales).

1. El código implementa un decodificador de 2 a 7 bits, donde según el valor binario de la entrada A, se genera una combinación específica de señales en sal.
2. Es muy probable que sal esté conectado a un display de 7 segmentos, de modo que cada combinación enciende ciertos segmentos para representar un número o letra.

COMPILACIÓN:

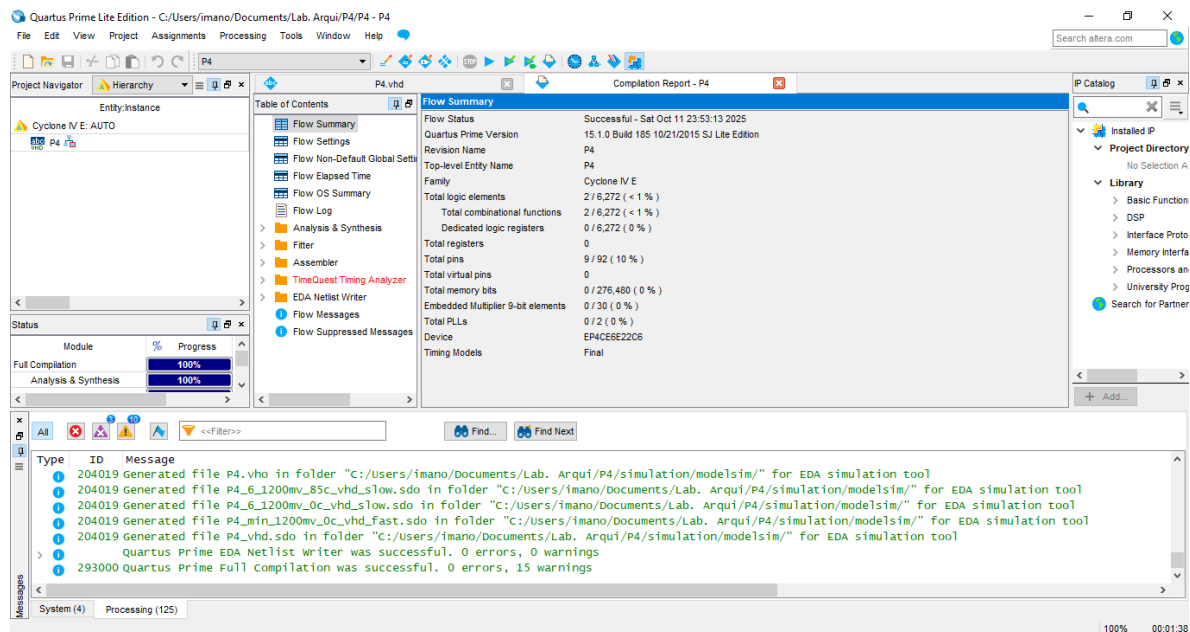


Imagen: Compilación

SIMULACIÓN 1

Como quiero hacer la simulación en Waveform Editor, entonces seguimos el siguiente paso. En la ventana New, en la sección Verification/Debugging Files seleccionar la opción University Program VWF.

- A_0 con periodo de 20 ns,
- A_1 con periodo de 40 ns,

Entonces estás generando una **entrada binaria de 2 bits (A1 A0)** que cambia automáticamente en el tiempo, produciendo los 4 posibles valores de A.

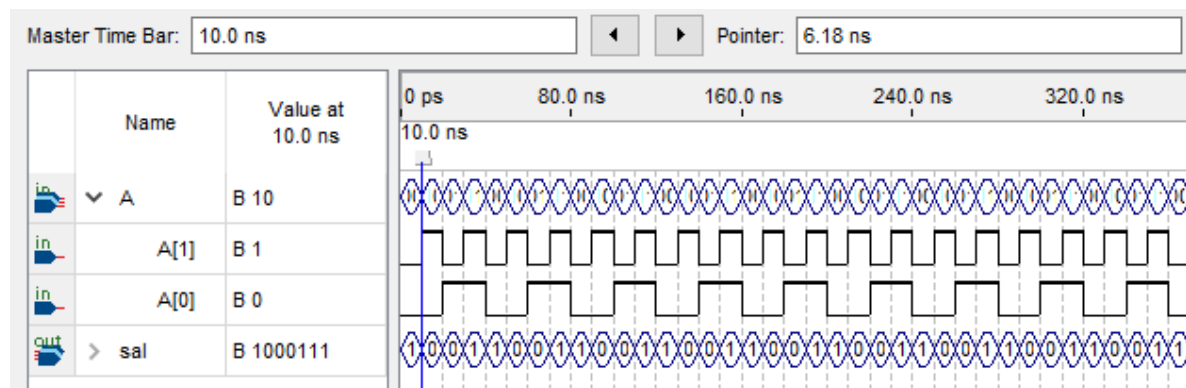


Imagen: Simulación

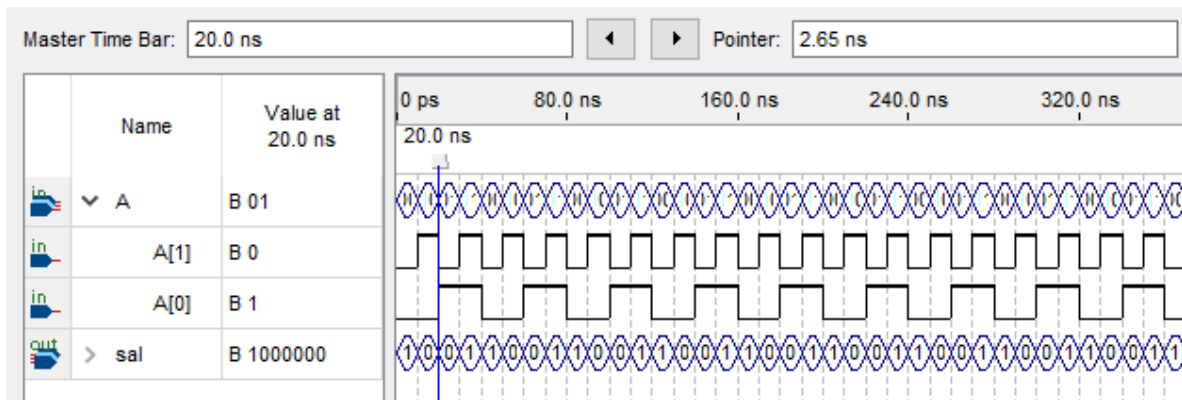


Imagen: Simulación

Dado que:

- A_0 cambia cada **10 ns** (porque su periodo es 20 ns, y alterna entre 0 y 1 cada mitad),
- A_1 cambia cada **20 ns** (porque su periodo es 40 ns),

se obtiene la siguiente secuencia de combinaciones binarias en el tiempo:

Tiempo (ns)	A_1	A_0	Valor de A (A_1A_0)
0–10	0	0	"00"
10–20	0	1	"01"
20–30	1	0	"10"
30–40	1	1	"11"
40–50	0	0	"00"
(y se repite)			

Así, cada **40 ns** se completa un ciclo completo de los 4 valores posibles de A.

Tu proceso usa un **case** sobre A, así que sal tomará distintos valores según esa tabla.

A (A_1A_0)	Valor de sal
"00"	"0001001"
"01"	"1000000"
"10"	"1000111"
"11"	"0001000"

SIMULACIÓN 2

Utilizamos la otra simulación que es la misma interpretación

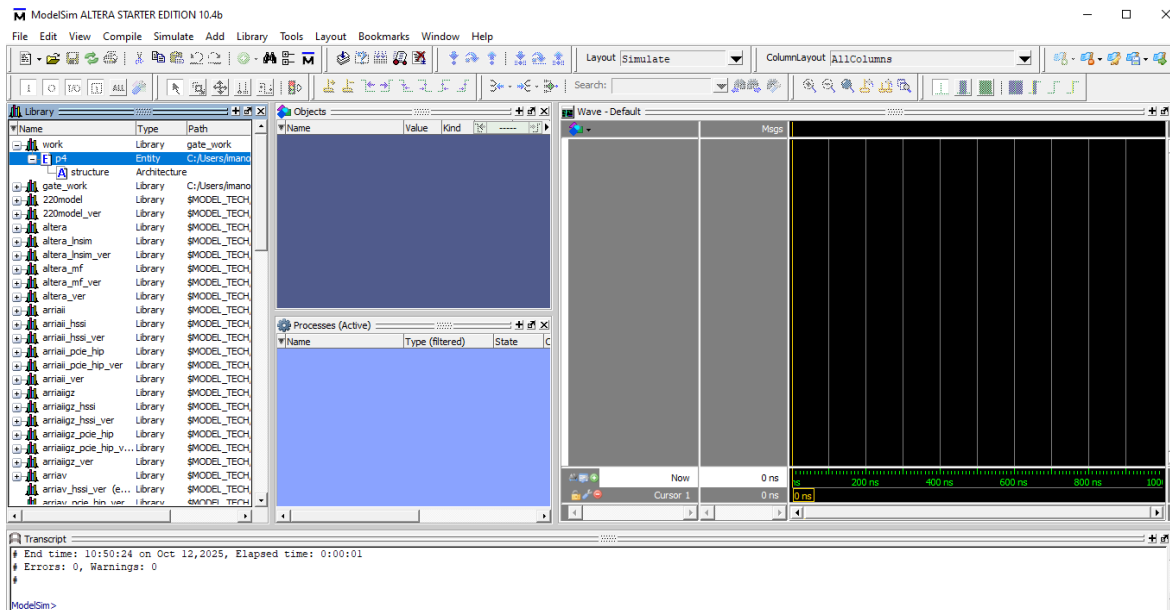


Imagen: Simulación

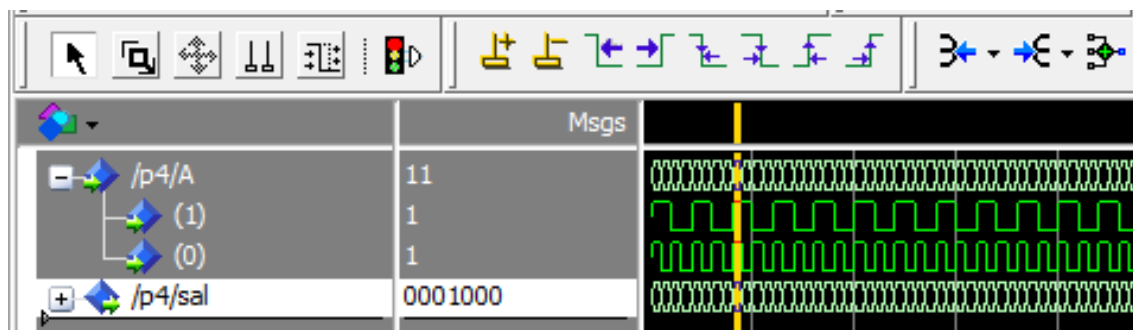


Imagen: Simulación

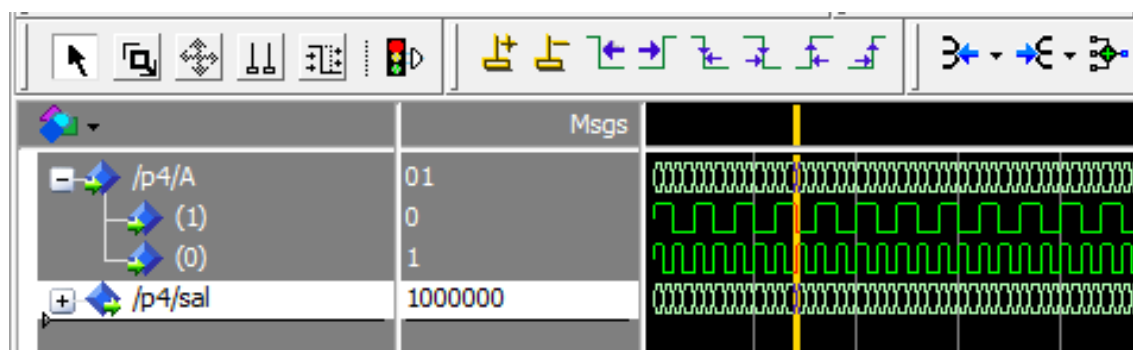


Imagen: Simulación

Por tanto, durante la simulación:

Tiempo (ns)	A (A1A0)	sal
0–10	"00"	"0001001"
10–20	"01"	"1000000"
20–30	"10"	"1000111"
30–40	"11"	"0001000"
40–50	"00"	"0001001"
<i>(y se repite cada 40 ns)</i>		

Lo que sucede:

- La entrada A cambia automáticamente con el tiempo debido a los periodos distintos de A_0 y A_1 .
- La salida sal responde instantáneamente a esos cambios, tomando los valores definidos en el case.
- Se genera una **secuencia cíclica de 4 estados** que se repite cada **40 ns**.

Visualmente (en una simulación o display):

- Verías que sal cambia cada 10 ns, recorriendo los cuatro patrones predefinidos.
- Si sal controla un display de 7 segmentos, éste mostraría **4 figuras o números distintos** que se van alternando rápidamente (cada 10 ns).